

ADR-004: The Custody Server — the Laboratory Bench on Paper

Status: Proposed — frozen for review before implementation · Date: 3 August 2026 · Author: Ken Tombs with Claude (design lead, Opus 4.8) · Governing design: ADR-003 (three-layer architecture; this ADR specifies Layer 2, the experimental environment / mediation layer). Companion: Design Rationale — Direct Access vs Curation. → reference (Master Reference: Architecture).

1. Context

ADR-003 established three layers — research subject, experimental environment, research method — with governance as an independent variable, and reserved Phase 2 for a mediation layer between the AI and the evidence. Phase 1 (Sessions 1–5) ran the subject with DIRECT filesystem access and policed custody by hand: the operator at every approval prompt, the audit bench for every verification, the logs written by discipline. Phase 1 proved the concept and measured its cost — governance overhead roughly tripled under the primer, each approval demanding operator expertise, alarm fatigue setting in. The lesson repeated across six sessions: rules that live in instructions and human attention degrade; the durable answer is infrastructure that enforces policy structurally.

This ADR specifies that infrastructure: an MCP (Model Context Protocol) server sitting between Claude Code and the evidence, exposing a fixed set of operations, enforcing scope and integrity, and generating a custody record independent of anything the AI reports about itself. In Leonard’s formulation it is the laboratory bench: it provides controlled conditions; it does not perform the chemistry. It contains policy, not intelligence.

2. Decision

We adopt a six-tool, read-and-verify MCP custody server — "Dossier Space MCP Server v1" — operating in a RECORDED-DIRECT posture: the subject reaches the raw source directly (highest evidential quality), every access is hashed and logged at the moment it occurs, scope is enforced by filter, and provenance divergence is flagged automatically. The server has no write, edit, or delete capability in its vocabulary — read-only is enforced by OMISSION, not by a rule that could be bypassed. The tool set and its definitions are frozen by this ADR before any code is written, so the custody guarantees can be reviewed independently of the implementation.

3. The tool set — and why exactly these six

The set must be complete enough to do real evidence work and minimal enough that every operation is trivially auditable. Six earns that balance. Note the shape: the set is entirely read-and-verify; the last three tools (get_headers, hash_document, verify_hash) are what make this an EVIDENCE server rather than a generic file bridge — they exist to serve custody and citability.

Tool	Inputs → output	Purpose and why it belongs in the minimal set
list_folder	scoped path → names, sizes, dates	Enumerate what exists in a scoped location — the subject’s map of the evidence. Returns metadata only, never content.
read_file	one scoped file path → content + access record	The atomic retrieval operation. Returns one named file’s content, and writes a hashed access record as it does (see §5). The only way the subject ever obtains evidence content.
search_messages	scope + criteria → matching file list	Find files matching keyword, date, or custodian WITHOUT reading them all — the scaling operation. Without it the subject would read half a million files to answer one question. Returns paths, not content.

Tool	Inputs → output	Purpose and why it belongs in the minimal set
get_headers	one file path → header fields only	Extract metadata (From, To, Date, Subject, Message-ID) without the body — the exhibit-identifier operation. Directly serves the exhibit discipline the primer restores: unique identifiers for citable work.
hash_document	one file path → SHA-256	The integrity operation. Every claim the subject makes about a file can be anchored to a verifiable content hash, computed by infrastructure rather than asserted by the AI.
verify_hash	file path + expected hash → match / divergence	The tamper-detection operation. Confirms a file still matches a previously recorded hash — proof the evidence has not changed since last examined. The active half of the custody guarantee.

There is no `write_file`, `edit_file`, or `delete_file`. This is deliberate and load-bearing: the server cannot modify evidence because the capability does not exist in its vocabulary. Read-only enforced by omission cannot be bypassed by instruction, jailbreak, or error, because there is nothing to bypass. The subject writes its **OUTPUTS** through its own client to its own outputs folder, entirely outside this server — the server's world is the read-only evidence, and only that.

4. Scope — the privilege gate as filter, not wall

Per the Design Rationale (Direct Access vs Curation), the server does NOT serve from a pre-approved manifest — curation institutionalises an unverifiable sense-check and launders the corpus's state-at-approval as authentic. Instead the privilege gate is a **SCOPE FILTER** declared per matter: permitted custodians, date ranges, and folder paths. A request outside declared scope is refused and the refusal logged. The gate is a boundary on what the subject may reach, not a curated list of what is "correct" — the subject reaches the raw source directly within the declared boundary, and the source is interrogable rather than asserted.

Scope is declared in a matter configuration file, human-authored, version-controlled, and frozen at the start of a work phase (changing it is a logged, deliberate act — the same discipline as changing a tool description). A dedicated privilege list may additionally name custodians or folders the server will **REFUSE** regardless of any request — the discovery-practitioner's privilege gate, for material that must never be served (legally privileged, out-of-scope-by-order, sealed).

5. The custody log — recorded-direct, chained, tamper-evident

Every operation writes one log entry at the moment of execution, and each entry incorporates the hash of the previous entry — a chain, so the record is tamper-EVIDENT (any alteration breaks the chain from that point forward), not merely tamper-resistant. An entry records:

- sequence number and timestamp (server clock, recorded)
- tool called, and its exact inputs (scoped path, criteria)
- for `read_file` / `get_headers`: SHA-256 of the file AS READ, at read time
- result summary (bytes returned, match/divergence, refusal + reason)
- provenance flag, if any (see §6)
- hash of the previous log entry (the chain link)

This is the recorded-direct posture made concrete: the subject reaches raw source directly, but the log captures the evidence **EXACTLY AS IT WAS** when the AI touched it. Manipulation before access surfaces as divergence against independently-held prior state; manipulation after surfaces in `verify_hash`. The server never decides authenticity; it records what it saw, precisely, so authenticity can be argued from evidence later. The log is the infrastructure-side custody trail — independent of, and comparable against, the subject's own ROMER self-report. The gap between the two (what the AI says it did vs what

the server recorded it doing) is itself a measurable finding — the trace-versus-log gap ADR-003 reserved for measurement, now instrumentable for the first time.

6. Provenance flagging — baked in, automatic, flag-don't-curate

DESIGN DECISION (design-lead recommendation, adopted by default; reversible on review): provenance-divergence flagging is a BEHAVIOUR baked into the logging layer, running automatically at access — NOT a tool the AI calls. Rationale: an integrity check the subject can decline to exercise is not a guarantee; provenance checking that depends on the AI choosing to run it reintroduces exactly the discretion the custody layer exists to remove. The infrastructure vouches, not the analyst.

On access, the server compares the file's current state against any independently-held prior baseline it has been given (a pre-existing manifest, a recovered hash set) and FLAGS divergence in the log — it does not block, and it does not correct. The distinction is the whole philosophy: a wall would silently refuse; a flag reports and lets the humans reason about it. The Enron corpus is the live proof: it WAS manipulated (the 3 July .eml rename of ~517,000 files), and we DO hold the pre-manipulation baseline (manifest.txt, 1 July; the recovered git commit declaring the CMU source). The honest server flags "current filenames diverge from manifest.txt — a transformation has occurred here" rather than working the renamed state as though it were original. A server that flags its own corpus's rename is doing exactly what evidence work demands: it does not sense-check the evidence into looking correct; it reports what it can and cannot vouch for.

7. Self-describing tools — and the experimental caution

Each tool carries a description (name, purpose, permissions, that its use is logged) transmitted to the model via MCP. Two things follow. Legally: the descriptions make the server's policy legible — a reviewer can read what each tool is permitted to do. Experimentally: in MCP, descriptions are sent to the model and SHAPE ITS BEHAVIOUR — they are part of the experimental environment, not documentation beside it. They must therefore be frozen for the duration of any experiment, recorded verbatim in the method, and published with results. Changing a tool description mid-experiment silently changes the treatment. Legally-reviewable and behaviourally-influential are the same artefact here; that dual character is stated so no reviewer misses it.

8. Phasing — how this server is measured (per ADR-003)

Phase 2a (passthrough / null-mediation): the server serves the six tools with hashing and chained logging switched OFF — pure mediation, a passthrough. Measures the cost of INDIRECTION alone.

Phase 2b (custody on): the same server with hashing, chained logging, scope enforcement, and provenance flagging active. The 2a–2b delta isolates the cost of GOVERNANCE specifically — the number the paper reports. Both phases run the same tasks as Phase 1 (survey, retrieval), scored against the same ground truth, comparable to the Phase 1 grid. The trace-versus-log gap is measured in 2b.

9. Consequences

Positive: read-only enforced by omission (unbypassable); custody generated by infrastructure independent of the subject's self-report; the raw source reached directly (highest evidential quality) without sacrificing the custody trail; manipulation surfaced rather than assumed away; scope and privilege enforced structurally; the server a citable, versioned artefact ("Dossier Space MCP Server v1");

the trace-versus-log gap finally instrumentable. The design honours the principal's insight that interrogable raw source beats curated access.

Negative and accepted: the subject's outputs still travel to the provider for inference — the server governs access and custody, not data locality (the sovereignty caveat stands; a local-inference variant plugs the same server behind a different engine). The scope filter is a boundary, not a curation — it does not vouch that in-scope files are authentic, only that out-of-scope files are not served; authenticity is argued from the log, which is the intended design, not a gap. Provenance flagging is only as good as the baselines it is given — where no prior state exists, the server records what it sees as the earliest baseline and says so.

10. Actions

Freeze this tool set and the (to-be-written) verbatim tool descriptions; author the matter scope-configuration format and the privilege-refusal list; specify the chained-log entry format precisely enough to implement; implement Dossier Space MCP Server v1 in Python against this ADR (Session 8), starting in 2a passthrough; supply the server the Enron baselines (manifest.txt, recovered git hashes) so provenance flagging can be demonstrated against the known rename; then the measured 2a/2b runs (Session 9). Scorecard rubric for Session 10 to be drafted alongside, criteria fixed before the runs it will score.